Backtesting Probability of Default Models

The Two-Sided Exact Binomial Test

Andrija Djurovic

www.linkedin.com/in/andrija-djurovic

Internal Ratings-Based Modeling Process

- The development of an Internal Ratings-Based (IRB) model consists of two main areas: the development of the risk differentiation function and risk quantification.
- Development of the risk differentiation function is the first step in the overall model development process. The main goal of this step is to ensure the model can adequately rank risk, providing meaningful differentiation between low- and high-risk obligors or facilities.
- After developing the risk differentiation function, practitioners perform the risk quantification process, which essentially involves assigning an appropriate risk level to the output of the risk differentiation function.
- Risk quantification typically consists of specific steps such as: assignment of the base or best estimate, quantification of appropriate adjustments (AA), quantification of the margin of conservatism (MoC), and quantification of any other additional conservatism, for instance that required by certain local regulators. The first two steps are typically referred to as calibration.
- These steps apply to all three risk parameters: Probability of Default (PD), Loss Given Default (LGD), and Exposure at Default (EAD).

Probability of Default Backtesting

- As risk quantification is a regulatory requirement, practitioners pay significant attention to complying with these requirements.
- In addition to adhering to the prescribed risk quantification process, practitioners are required to assess the appropriateness of the model estimates. This is typically referred to as backtesting.
- The main purpose of backtesting in this context is to ensure that the model estimates adequately predict the occurrence of defaults, i.e., that PD estimates constitute reliable forecasts of default rates.
- In practice, this is one of the most comprehensive validation exercises, typically investigating model estimates across different portfolio subsegments and time dimensions.
- One commonly applied approach is to backtest estimates over a recent period on a year-by-year basis.

Probability of Default Backtesting cont.

- In practice, three statistical tests are typically employed for backtesting estimates at the level of individual years: the exact binomial test, the z-score test, and Jeffreys' test.
- The exact binomial test relies on the (discrete) binomial distribution of the number of defaults, while the z-score and Jeffreys' tests rely on a continuous approximation of the binomial proportion distribution.
- The z-score test is traditionally used in both one-sided and two-sided versions, whereas the exact binomial test is typically perceived by practitioners as lacking a natural extension to a two-sided version. While this can be considered true in the standard framework of hypothesis testing, a two-sided p-value from the exact binomial test can nevertheless be calculated.
- The purpose of this presentation is to demonstrate the calculation of the p-value of the two-sided exact binomial test. The following slide introduces the basics of the exact binomial test and its application in the backtesting of PD models, while the remainder of the presentation provides `R` and `Python` code for calculating the p-value.
- As all tests used for backtesting PD models (including versions that assume correlated defaults) rely on specific assumptions and generally align to varying degrees with the IRB framework, practitioners are encouraged to consider these aspects when using and interpreting the test results.

The Exact Binomial Test

The exact binomial test is frequently used to assess the backtesting of PD models. It is typically applied to test for underestimation of PD by defining a critical value for the number of defaults at a selected confidence level. Formally, for a confidence level q, the critical value of the test is given by:

$$k^* = min\left\{ k \mid \sum_{i=k}^{n} \binom{n}{i} PD^i (1 - PD)^{n-i} \leq 1 - q \right\}$$

where:

- n is the number of exposures;
- PD is the predicted PD.

The calculated critical value is then compared to the observed number of defaults, and a conclusion is drawn.

The Exact Binomial Test cont.

To align with the traditional hypothesis testing framework and the use of p-values, the exact binomial test for underestimation can be interpreted in terms of probability as follows:

$$Pr(x \geq nd) = \binom{n}{nd} PD^{nd}(1 - PD)^{n-nd}$$

where:

- n is is the number of exposures;
- nd is the number of defaults;
- PD is the probability of default.

Note:

The above approach forms the basis for the two-sided p-value of the exact binomial test. Essentially, in addition to the upper-tail probability, practitioners need to consider the lower-tail probability for the two-sided version.

The following slides demonstrate this calculation in `R` and `Python` using built-in functions, along with a step-by-step manual replication of the results. Practitioners should note that the manual calculation assumes the observed number of defaults (`nd`) is greater than the expected number of defaults (`pd*n`); for the opposite case, the calculation should be adjusted accordingly.

Finally, practitioners are encouraged to compare the results with the z-score test to better understand the differences and their impact on the conclusions of the testing procedure.

R Example

```r
#calibrated pd
pd <- 0.025
#observed number of defaults
nd <- 85
#sample size
n <- 3000

#bulit-in function
binom.test(x = nd,
           n = n,
           p = pd,
           alternative = "two.sided")
```

```
##
##  Exact binomial test
##
## data:  nd and n
## number of successes = 85, number of trials = 3000, p-value = 0.2418
## alternative hypothesis: true probability of success is not equal to 0.025
## 95 percent confidence interval:
##  0.02269297 0.03491667
## sample estimates:
## probability of success
##             0.02833333
```

```r
#manual calculation
rel.err <- 1 + 1e-07
d <- dbinom(x = nd, size = n, prob = pd)
m <- n * pd
i <- seq.int(from = 0, to = floor(m))
y <- sum(dbinom(x = i, size = n, p = pd) <= d * rel.err)
#p-value (sum of lower- and upper-tail probabilities)
pbinom(q = y - 1,  size = n, p = pd) +
pbinom(q = nd - 1, size = n, p = pd, lower.tail = FALSE)
```

```
## [1] 0.2418246
```

Python Example

```
import numpy as np
from scipy.stats import binom, binomtest

#calibrated pd
pd = 0.025
#observed number of defaults
nd = 85
#sample size
n = 3000

#bulit-in function
binomtest(k = nd,
          n = n,
          p = pd,
          alternative = "two-sided")
```

```
## BinomTestResult(k=85, n=3000, alternative='two-sided', statistic=0.028333333333333332, pvalue=0.2418245831784428)
```

```
#manual calculation
rel_err = 1 + 1e-07
d = binom.pmf(k = nd, n = n, p = pd)
m = n * pd
i = np.arange(0, int(np.floor(m)) + 1)
y = np.sum(binom.pmf(k = i, n = n, p = pd) <= d * rel_err)
#p-value (sum of lower- and upper-tail probabilities)
(binom.cdf(k = y - 1, n = n, p = pd) +
 binom.sf(k = nd - 1, n = n, p = pd))
```

```
## np.float64(0.2418245831784428)
```